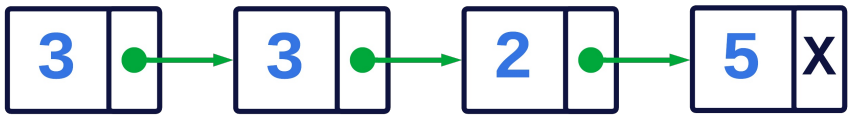
Learning Objectives

Learners will be able to...

- **Define a linked list and its components**
- **Implement a linked list in C++**
- **Apply different operations to a linked list, such as insertion, deletion, searching, and traversal**
- **Analyze the complexities of different linked list operations**

Linked List

After exploring array and vector-based lists, it's time to venture into another type of list that offers a different approach to storing and managing data: **linked lists**. Unlike arrays, which use a contiguous block of memory, linked lists are composed of individual elements known as **nodes**, each pointing to the next node in the list.



Our goal aims to demystify the concept of linked lists, focusing on their structure, types, basic operations, and use cases. We will also compare linked lists to arrays, shedding light on their respective strengths and weaknesses.

What is a Linked List?

A **linked list** is a linear data structure made up of nodes, where each node contains data and a reference (or link) to the next node in the list.

```
class Node {
public:
    int data; // the data part
    Node* next; // the reference to the next node
};
```

Types of Linked Lists

There are several types of linked lists, including:

1. **Singly Linked List**: Each node points to the next node.
2. **Doubly Linked List**: Each node points to both the next and the previous nodes.
3. **Circular Linked List**: The last node points back to the first node.

This assignment will only deal with singly linked lists. The other variations will be covered in subsequent assignments.

Basic Operations

Basic operations associated with linked lists include:

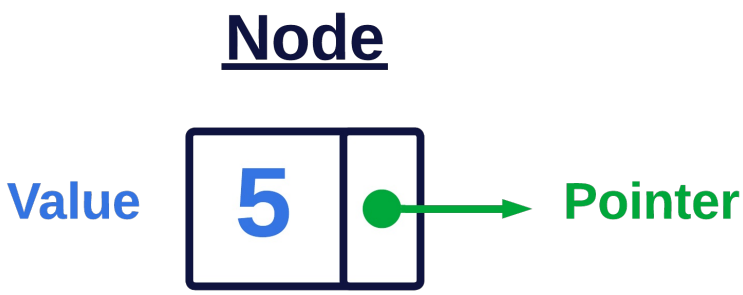
- **Insertion**: To add a new node.
- **Deletion**: To remove a node.
- **Traversal**: To go through the list.
- **Searching**: To find a node in the list.

Components of a Linked List

Breaking Down Linked Lists

When interacting with linked lists you will often hear the following terms: **value**, **pointers**, **head**, **tail**. It is important to understand these terms in order to grasp how linked lists function and how they perform operations like insertion, deletion, and traversal efficiently. The examples that follow use user-defined objects to define nodes and a linked list.

Nodes



For our example, nodes will be represented by the `Node` class. Just as a list is made up of elements, a linked list is made up of nodes. In a list, the job of an element is to store data. In a linked list, a node stores information and points to the next node in the list.

When we talk about **value**, we are talking about the information stored in the node. In our example, the value is represented by the `data` field.

Nodes, like lists, have a data type. The example on the opposite page is a node that works with integers. When you instantiate a `Node` object, the value passed to the constructor will be stored in the `data` field.

In the context of a linked list, a **pointer** is simply a reference to another node. The `next` field in our `Node` class acts like a pointer pointing to the next node in the list. As such, the `next` field is another `Node` object.

Every time that you add a new node to a linked list, it goes at the end. One of the defining characteristics of the last node is that it does not point to another node. Be sure that you instantiate a `Node` object with `nullptr` as the value for `next`.

Linked Lists



Now that we have seen a node, we can talk about a linked list as a whole. Linked lists are dynamic, and its size is often unknown. However, we can say with certainty that every linked list has a first and last node.

The **head** is the first node in the linked list. When a linked list is empty, the head is `nullptr`. When a new element is added to an empty list, the head points to this new element. The head is crucial for the traversal of the linked list; starting at the head, you can reach any node by following the `next` pointers.

The **tail** is the last node in the linked list. Unlike arrays, the tail in a singly linked list is not directly accessible; you have to traverse the list from the head to find it. However, in a doubly linked list or a circular linked list, you can keep a reference to the tail for quick access. The tail's `next` pointer is always `nullptr` in a singly linked list, signifying the end of the list.

▼ **Tails in other linked lists**

> In a singly linked list, there is no reference to the tail node. We can distinguish it because the `next` field has the value `nullptr`. As mentioned above, you may see a reference to the tail node in other variations of a linked list. It would look something like this:

```
class LinkedList {
    Node* head;
    Node* tail;

    // class code
};
```

Implementation

Coding a Linked List

Taking what we know, let's build a linked list. We will start with the `Node` class, and then create the `LinkedList` class. For now, we are going to keep things simple. Our linked list will work with integers. We will be able to add a node to the list, and then print the values stored in the list. We will discuss other operations later.

The `Node` class is used to represent each element in a linked list. It has two attributes, `data` and `next`. In the constructor, set the value of `data` to the constructor's parameter. By default, the `next` attribute should be `nullptr`.

```
class Node {
public:
    int data;  // Value stored in the node
    Node* next; // Reference to the next node in the linked list

    // Constructor for initializing a Node with a given data
        value
    Node(int data) {
        this->data = data;
        this->next = nullptr;  // Initially, the node doesn't
        point to any other node
    }
};
```

Next, we need the `LinkedList` class to encapsulate the behavior of our linked list. We need only one attribute, `head`, which is a `Node` object. This will represent the first node in our linked list. We are going to use C++'s default constructor, which initializes the `head` pointer to `nullptr`. The value for `data` will be `0` by default. Creating a `LinkedList` object will create a `node`, but it does not have a value or a pointer. That job is left to the `add` method.

```
class LinkedList {
public:
    Node* head;  // The head node of the linked list

    // Constructor for initializing an empty linked list
    LinkedList() {
        head = nullptr;
    }

    // Method to add a new node to the end of the linked list
    void add(int data) {
        Node* newNode = new Node(data);  // Create a new node
        with the given data


    }

};
```

We need to be able to `add` a new node to the list. Create the `add` method, which takes an integer as a parameter. Create a new `Node` object with the value from the parameter. We then need to place the new `node` at the end of the list. There are two possible cases, there are no nodes in the list (the head node has the value `nullptr`) or there are one or more nodes in the list. First check if the first node is `nullptr`. If so, set `head` to the new node.

```
    // Method to add a new node to the end of the linked list
    void add(int data) {
        Node* newNode = new Node(data);  // Create a new node
        with the given data

        // If the linked list is empty, make the new node the
        head
        if (head == nullptr) {
            head = newNode;
        }
    }
```

If there are `nodes` in the list, then we need to traverse it to find the end. Create a temporary node and assign its value to head. Using a while loop, check to see if the pointer for the temporary node is not `nullptr`. In this case, set the value of the temporary node to the next node in the list. Eventually, `temp->next` will be equal to `nullptr`. That means we have reached the end of the list. In this case, set `temp->next` to the new node.

```
    // Method to add a new node to the end of the linked list
    void add(int data) {
        Node* newNode = new Node(data);  // Create a new node
        with the given data

        // If the linked list is empty, make the new node the
        head
        if (head == nullptr) {
            head = newNode;
        } else {
            // Otherwise, traverse to the last node and add the
        new node there
            Node* temp = head;
            while (temp->next != nullptr) {
                temp = temp->next;  // Go to the next node
            }
            temp->next = newNode;  // Make the last node point
        to the new node
        }
    }
```

The last thing we want to do for this example is to print the linked list so we can see that our values are properly stored in the list. Create the `printList` method. Similar to the `add` method, we are going to traverse the list. Create a temporary node whose initial value is the first node in the linked list. Print the value for the `data` attribute, and then move the temporary node to the next node in the list. Once we reach the end. print a newline character.

```
    // Method to print all the elements in the linked list
    void printList() {
        Node* temp = head;  // Start from the head node
        while (temp != nullptr) {
            std::cout << temp->data << " ";  // Print the data
        of each node
            temp = temp->next;  // Go to the next node
        }
        std::cout << std::endl;  // Print a newline after
        traversing all nodes
    }
```

Create the `main` method so that you instantiate a `LinkedList` object and add `1`, `3`, `5`, `7`, and `9` to the end of the list. Then print the linked list.

```
int main() {
    LinkedList list;

    // Add elements
    list.add(1);
    list.add(3);
    list.add(5);
    list.add(7);
    list.add(9);

    list.printList();

    return 0;
}
```

▼ **Code**

> Your code should look like this:

```
class Node {
public:
    int data;  // Value stored in the node
    Node* next; // Reference to the next node in the linked
        list

    // Constructor for initializing a Node with a given
        data value
    Node(int data) {
        this->data = data;
        this->next = nullptr;  // Initially, the node
        doesn't point to any other node
    }
};

class LinkedList {
public:
    Node* head;  // The head node of the linked list

    // Constructor for initializing an empty linked list
    LinkedList() {
        head = nullptr;
    }

    // Method to add a new node to the end of the linked
        list
    // Method to add a new node to the end of the linked
        list
    void add(int data) {
        Node* newNode = new Node(data);  // Create a new
        node with the given data

        // If the linked list is empty, make the new node
        the head
        if (head == nullptr) {
            head = newNode;
        } else {
```

```
            // Otherwise, traverse to the last node and add
        the new node there
            Node* temp = head;
            while (temp->next != nullptr) {
                temp = temp->next;  // Go to the next node
            }
            temp->next = newNode;  // Make the last node
        point to the new node
        }
    }

    // Method to print all the elements in the linked list
    void printList() {
        Node* temp = head;  // Start from the head node
        while (temp != nullptr) {
            std::cout << temp->data << " ";  // Print the
        data of each node
            temp = temp->next;  // Go to the next node
        }
        std::cout << std::endl;  // Print a newline after
        traversing all nodes
    }

};

int main() {
    LinkedList list;

    // Add elements
    list.add(1);
    list.add(3);
    list.add(5);
    list.add(7);
    list.add(9);

    list.printList();

    return 0;
}
```

challenge

Try this variation:

Update the `main` method so that you instantiate a `LinkedList` object but do not add any nodes. Then print it.

```
int main() {
    LinkedList list;

    // Print list
    std::cout << "Linked List after adding elements:" <<
        std::endl;
    list.printList();

    return 0;
}
```

▼ **What is happening?**

> C++'s default constructor should create the `head` and give it the value of `nullptr`. Yet, the program does not print anything from the linked list. Why? If you look at the `printList` method, you will notice that the while loop only runs if the temporary node is not `nullptr`. The default constructor will set the value of `head` to `nullptr`, so the loop never runs.

Linked List Operations

Basic Operations

Having established a fundamental understanding of pointers, the head, and the tail of a linked list, let's now look at some basic operations like `traversal`, `insertion`, `deletion`, and `searching`. We will use our previously defined `Node` and `LinkedList` classes for this purpose.

Traversal

Traversing a linked list involves walking through the list from the head to the last node. We have seen examples of this with the `add` and `printList` methods. Linked lists do not have the `size` method or `length` attribute. We cannot know the length of any given linked list ahead of time, so we need to utilize a while loop. Create a temporary node. This will act as a pointer to each node in the list. Start with the head node, and as long as the temporary node is not `nullptr`, advance to the next node by setting the value of the temporary node to the pointer of the current node. Add the following method to the `LinkedList` class.

```
int countNodes() {
    Node* temp = head;
    int counter = 0;
    while (temp != nullptr) {
        counter += 1;
        temp = temp->next;
    }
    return counter;
}
```

After you add the `countNodes` method to the `LinkedList` class, update the `main` method to display the number of nodes in the list.

```
int main() {
    LinkedList list;

    // Add elements
    list.add(1);
    list.add(2);
    list.add(3);

    // Count nodes
    std::cout << "There are " << list.countNodes() << " nodes."
        << std::endl;

    return 0;
}
```

Insertion at the Beginning

To insert a node at the beginning of the list, you have to change the head to the new node and make the new node point to the original head. Add the following method to the `LinkedList` class.

```
  void insertAtBeginning(int data) {
    Node* newNode = new Node(data);
    newNode->next = head;
    head = newNode;
  }
```

Update the `main` method to insert a new node with the value of `4` at the beginning of the list. Use the `printList` method to verify that it works as expected.

```
  int main() {
    LinkedList list;

    // Add elements
    list.add(1);
    list.add(2);
    list.add(3);
    list.insertAtBeginning(4);

    // Print nodes
    std::cout << "Linked List after adding elements:" <<
        std::endl;
    list.printList();

    return 0;
  }
```

Insertion at the End

To add a node at the end of the list, you have to traverse the list to the last node and then link the new node. The logic for this already exists in the `add` method, however `insertAtEnd` does a better job at describing what the method does. Replace the `add` method with the one below.

```
void insertAtEnd(int data) {
    Node* newNode = new Node(data);
    if (head == nullptr) {
        head = newNode;
        return;
    }
    Node* last = head;
    while (last->next != nullptr) {
        last = last->next;
    }
    last->next = newNode;
}
```

Update the `main` method so that it calls `insertAtEnd` instead of `add`.

```
  int main() {
    LinkedList list;

    // Add elements at the end
    list.insertAtEnd(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);

    // Print list
    std::cout << "Linked List after adding elements:" <<
        std::endl;
    list.printList();

    return 0;
  }
```

Deletion

To delete a node, you first need to find it and then rearrange the pointers to exclude it from the list. There are three different cases for this operation. The node to delete is the head, the node to delete is not in the list, or the node to delete is somewhere in the list. Let's take a look at how to write code for each of the cases. We are going to need two additional `Node` objects when performing a deletion – `temp` and `prev`.

First, check to see if the node to delete is the head. We know this to be true if value of the head is not `null` and the value in the node matches the value we want to delete. If so, set the `head` attribute to `temp->next`. Now, the `head` attribute points to the second node in the list.

```
void deleteNode(int data) {
    Node* temp = head, *prev = nullptr;
    if (temp != nullptr && temp->data == data) {
        head = temp->next;
        return;
    }
}
```

If the node to delete is not the head, then we need to traverse the list. In this case, we want to keep progressing through the list as long as `temp` is not `nullptr` and the value of the node is not the value we want to delete. When this loop stops, it could be for one of two reasons: either you have reached the end of the list, or you have found the value to delete. Notice how we iterate: the `temp` node moves to the next node in the list, while `prev` is one node behind `temp`.

```
void deleteNode(int data) {
    Node* temp = head, *prev = nullptr;
    if (temp != nullptr && temp->data == data) {
        head = temp->next;
        return;
    }
    while (temp != nullptr && temp->data != data) {
        prev = temp;
        temp = temp->next;
    }
}
```

If we are at the end of the list (`temp == nullptr`), that means the value to delete was not found. Use a return statement to exit the method. If the loop ended because you found the value to delete, set the value of next attribute of `prev` to the `next` attribute of the `temp` node. In effect, `prev` now points to the node after `temp`, which deletes the node since we will never see it again on a subsequent traversal.

```
void deleteNode(int data) {
    Node* temp = head;
    Node* prev = nullptr;

    if (temp != nullptr && temp->data == data) {
        head = temp->next;
        delete temp;
        return;
    }

    while (temp != nullptr && temp->data != data) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == nullptr) return;

    prev->next = temp->next;
    delete temp;
}
```

Update the main method to print the linked list before and after the deletion.

```
int main() {
    LinkedList list;

    // Add elements
    list.insertAtEnd(1);
    list.insertAtEnd(3);
    list.insertAtEnd(5);
    list.insertAtEnd(7);
    list.insertAtEnd(9);

    // delete node
    std::cout << "Linked List before deletion:" << std::endl;
    list.printList();

    list.deleteNode(7);

    std::cout << "Linked List after deletion:" << std::endl;
    list.printList();

    return 0;
}
```

Searching

To search for a node in a linked list, you are going to use a linear search. Start at the head and traverse until you find the element or reach the end of the list. Return `true` if you find the value, or `false` if you do not. Unlike searching an array, you do not return an index, as direct access to a value is not done in a linked list.

```
bool search(int data) {
    Node* current = head;
    while (current != nullptr) {
        if (current->data == data)
            return true;
        current = current->next;
    }
    return false;
}
```

Update the `main` method to search for two different values – one that is in the list and another that is not.

```
int main() {
    LinkedList list;

    // Add elements
    list.insertAtEnd(1);
    list.insertAtEnd(3);
    list.insertAtEnd(5);
    list.insertAtEnd(7);
    list.insertAtEnd(9);

    // search list
    std::cout << std::boolalpha; // Print boolean values as
        true/false
    std::cout << "Search for 3: " << list.search(3) <<
        std::endl;
    std::cout << "Search for 2: " << list.search(2) <<
        std::endl;

    return 0;
}
```

These basic operations form the core of linked list manipulation. Understanding how they work is essential for both utilizing linked lists effectively and for building more complex data structures based on linked lists.

Advantages and Disadvantages of Linked Lists

After covering the basics of linked list operations, it's essential to understand where and when to use linked lists. In this section, we will talk about the advantages and disadvantages of using linked lists compared to arrays, and highlight scenarios where one may be preferable over the other.

Advantages

Linked lists may not have the built-in methods and attributes of arrays and vectors, but their unique structure does provide the following benefits:

- **Dynamic Size**: Linked lists dynamically allocate memory, which is a significant advantage over arrays with fixed size.
- **Ease of Insertion/Deletion**: Inserting a new node or deleting a node is easier and faster compared to arrays as you just need to update some pointers. There's no need to shift elements.
- **No Wasted Space**: Because linked lists allocate memory as needed, they do not waste space like arrays can.

Disadvantages

Linked lists, however, have their own set of tradeoffs. Here are some reasons you may not want to implement a linked list:

- **Memory Usage**: Each node in a linked list requires extra memory for the `next` pointer, which can be a disadvantage for large lists.
- **Traversal**: Elements are stored in non-contiguous memory locations. Therefore, traversal can be slower than that of arrays.
- **Random Access**: Direct/random access is not allowed. One must traverse the list from the head node to find the desired element.

When to Use Linked Lists

Given the above information, here are some scenarios in which a linked list may be a good choice:

- **Unknown Size**: When you don't know the size of the list in advance.
- **Frequent Additions/Deletions**: When you have frequent insertions and deletions, and these operations need to be fast.
- **Stack/Queue Implementations**: They can be used to implement other data structures like stacks and queues.

You should avoid using a linked list when you need fast, random access to the values stored in the list. This operation relies too much on traversal to offer a fast solution. You should also consider not using a linked list when memory is a concern. While linked lists allocate memory as needed, storing the pointer to the next node does require some additional memory.

Code Comparison

Just for a recap, let's compare how one might implement insertion in an array and a linked list.

While both arrays and linked lists offer $O(1)$ insertion at the end, linked lists can provide $O(1)$ insertion at the beginning and in the middle, given that you have a reference to the node after which you want to insert.

Complexities of Linked List Operations

Analyzing Complexities of Linked List Operations

Having discussed the general advantages and disadvantages of linked lists, let's delve into the performance aspect, specifically the time complexity for different operations.

Time Complexities

- **Insertion at the Beginning**: $O(1)$ - Constant time as you only need to change a few pointers.
- **Insertion at the End**: $O(n)$ - You may need to traverse the list to insert at the end, unless you maintain a tail pointer.
- **Deletion**: $O(n)$ - In the worst case, you may have to traverse the list to find the node to be deleted.
- **Traversal**: $O(n)$ - You have to walk through the list from the head to the tail.
- **Searching**: $O(n)$ - Similar to traversal, you may need to go through each element in the list.

There are some best-case scenarios for linked lists. However, these scenarios require advanced knowledge of the node being acted upon. The only node that is known is the head, which is why insertion at the beginning is so quick. All other operations make use of traversal, which explains the $O(n)$ time complexity.

▼ **Keeping track of the tail**
Some implementations of a linked list will keep track of the tail. In these instances, you can quickly insert at the end of the list because no traversal is required. Not all linked lists, however, keep track of the tail node.

Space Complexity

The space complexity for a linked list is $O(n)$. In addition, each node takes up some additional space for storing the `next` pointer, aside from the data.

Comparing with Vector Operations

Let's take a look at some common operations and compare them when using linked lists and vectors.

Insertion

In a vector, you can have fast insertions by adding an element to the end of the list.

```
// insertion in a vector

int main() {
    std::vector<int> arr;
    arr.push_back(1);  // O(1) time complexity

    return 0;
}
```

You can also add elements to other locations in the vector. For example, `arr.insert(arr.begin(), 2)` adds the value `2` to the beginning of the list. However, this requires shifting of the vector, making insertion $O(n)$.

In a linked list, fast insertion happens at the beginning of the list since the head is always known.

```
// insertion in a linked list
struct Node {
    int data;
    Node* next;
};

int main() {
    Node* head = nullptr;

    Node* newNode = new Node;
    newNode->data = 2;
    newNode->next = head;
    head = newNode;  // O(1) time complexity

    return 0;
}
```

The lack of indices means that appending after any node with a given value requires traversal. Again, insertion goes from $O(1)$ to $O(n)$.

Deletion

We see a similar relationship between linked lists and vectors when it comes to deletion. The performance could be $O(1)$ under the right circumstance, or it can be $O(n)$. In the case of vectors, deletions are fast when removing the last element as there is no shifting that needs to be done.

Likewise, deletions are fast in a linked list when the node to remove is known (typically the head). However, if you do not know where the node is in the list, you have to rely on traversal to find it.

Searching

Vectors offer much more flexibility when it comes to searching. If you know the index, then searching is $O(1)$. If the target element is unknown, there are several searching algorithms that provide better performance than a simple linear search.

Linked lists, however, have one known node (head). There may be a few implementations keeping track of the tail, but these are not common. Known nodes have a time complexity of $O(1)$. Unknown nodes require traversal, which is $O(n)$.

In summary, linked lists offer more flexibility than vectors but come at the cost of additional memory and slower random access times. Your choice between the two should depend on the specific needs and constraints of your project.

Formative Assessment 1

Formative Assessment 2